BC01-FM04 · séance 3 · mardi 10 novembre 2026, matin

La liste d'emprunts · fiche de A

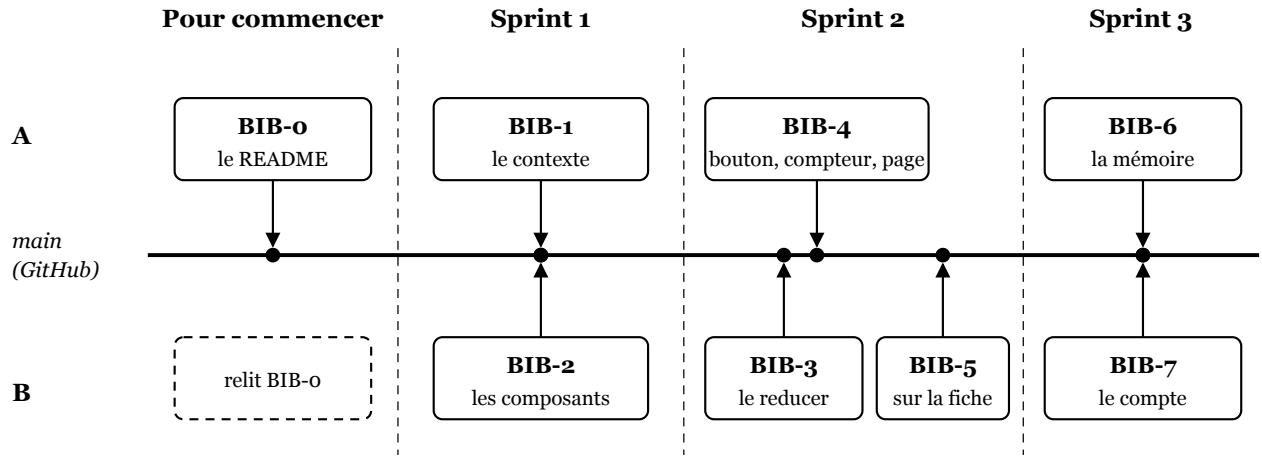
À rendre : `NOM1-NOM2-React-S3.zip`, le vendredi 13 novembre à 18 h 00
Avec : votre binôme, qui a la fiche B

Vous êtes deux développeurs. Le client veut une liste d'emprunts dans son catalogue. Le travail est découpé en tickets : chacun les siens, chacun sa branche.

Cette fiche dit **quoi faire**. Le **guide complet** explique **pourquoi** : ouvrez-le quand un morceau de code ou une commande vous échappe.

Vos tickets

Votre ticket	Ce que vous faites	B, en même temps
BIB-0	Le README de l'équipe	relit
BIB-1	Le contexte de connexion	BIB-2 : les composants qui le lisent
BIB-4	Le bouton, le compteur, la page « Ma liste »	BIB-3 : le reducer de la liste
BIB-6	La liste survit au rechargement	BIB-7 : le compte



Qui fait quoi, et quand. Chaque ticket rejoint main par une pull request, relue par l'autre.

Pour commencer : Git

1 · Votre poste

Dans le terminal de VS Code :

```
git --version
git config --global user.name "Prénom Nom"
git config --global user.email "vous@exemple.fr"
```

Pas de numéro de version ? Installez Git : **git-scm.com** sous Windows, `xcode-select --install` sous macOS. Il vous faut aussi un compte sur **github.com**.

---

2 · Le dépôt

Ouvrez le zip, et le dossier `catalogue-react/` dans VS Code.

```
git init
git add .
git commit -m "Code de départ"
git branch -M main
```

Sur GitHub : +, *New repository*, nom `catalogue-react`, *Public*, rien d'autre de coché.

```
git remote add origin https://github.com/VOTRE-IDENTIFIANT/catalogue-react.git
git push -u origin main
```

Vous devez voir : dans le terminal, une ligne qui finit par `main -> main`. Sur GitHub, vos fichiers apparaissent.

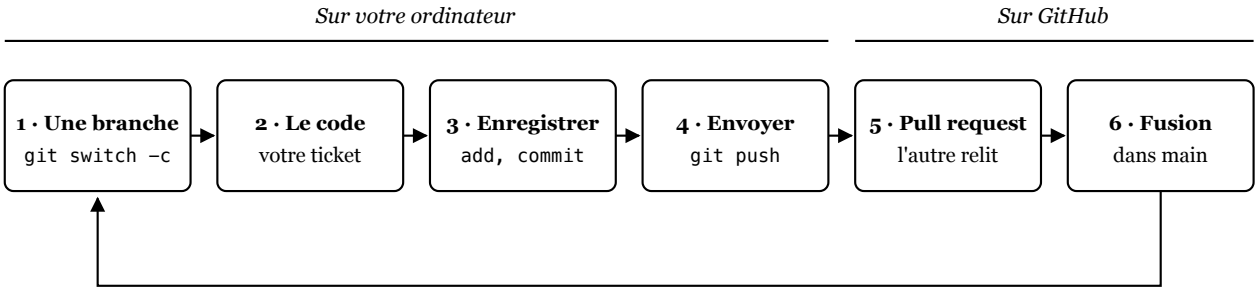
Invitez B : *Settings → Collaborators → Add people.* Puis lancez le projet :

```
npm install
npm run dev
```

Vous devez voir : le catalogue, à l'adresse http://localhost:5173.

3 · La boucle d'un ticket

La même pour tous vos tickets. Gardez cette page sous les yeux.



La boucle d'un ticket : trois gestes sur votre ordinateur, trois sur GitHub

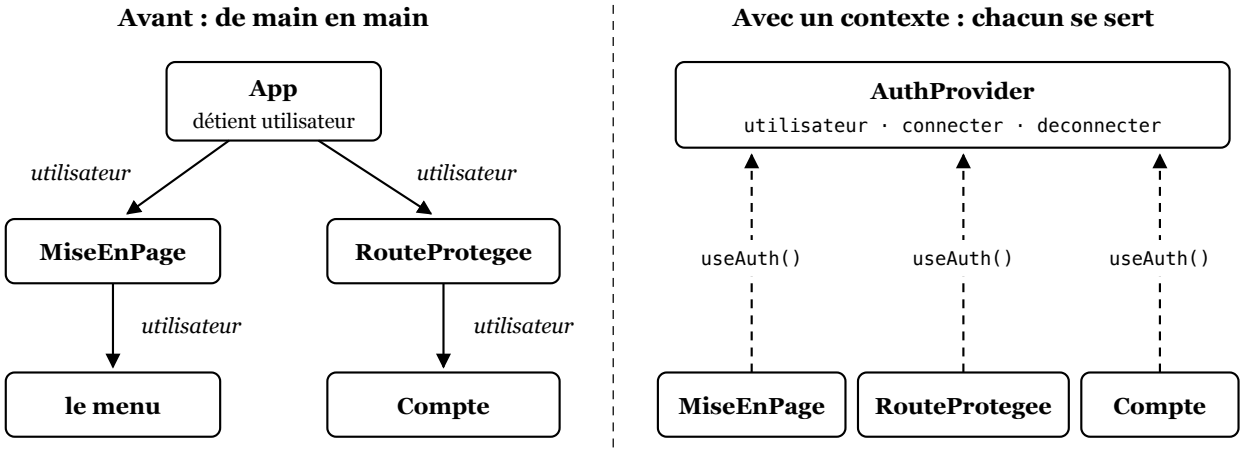
Quand	**Quoi**
Avant le ticket	`git switch main`, `git pull`, `git switch -c LA-BRANCHE`
Le ticket est fini	`git add .`, `git commit -m "LE MESSAGE"`, `git push -u origin LA-BRANCHE`
Sur GitHub	*Compare & pull request*, puis *Create pull request*
Votre binôme	lit *Files changed*, vous lui expliquez, il clique *Review changes → Approve*
Vous	*Merge pull request*, puis *Delete branch*
Tous les deux	`git switch main`, `git pull`

BIB-0 · Le premier aller-retour

Branche : `bib-0-readme`. Message : `BIB-0 : l'équipe et le dépôt`.

Dans `README.md`, complétez les noms de l'équipe et l'adresse du dépôt. Puis faites toute la boucle.

Sprint 1 · La connexion



Ce que change le sprint 1 : l'utilisateur ne descend plus de main en main

Le contrat : `useAuth()` renvoie `{ utilisateur, connecter, deconnecter }`.

BIB-1 · Le contexte de connexion

Branche : `bib-1-contexte-connexion`. Message : `BIB-1 : le contexte de connexion`.

Remplacez tout le contenu de `src/contexts/AuthContext.jsx` par :

```
import { createContext, useContext, useState } from 'react';

// Qui est connecté : une valeur que toute l'application peut lire, sans la passer
de main en main.
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [utilisateur, setUtilisateur] = useState(null);

  const valeur = {
    utilisateur,
    connecter: setUtilisateur,
    deconnecter: () => setUtilisateur(null),
  };

  return <AuthContext value={valeur}>{children}</AuthContext>;
}

export function useAuth() {
  const contexte = useContext(AuthContext);
  if (!contexte) {
    throw new Error('useAuth doit être utilisé dans un <AuthProvider>.');
  }
  return contexte;
}
```

Dans `src/main.jsx`, juste après la ligne `import { BrowserRouter } from 'react-router';` :

```
import { AuthProvider } from './contexts/AuthContext.jsx';
```

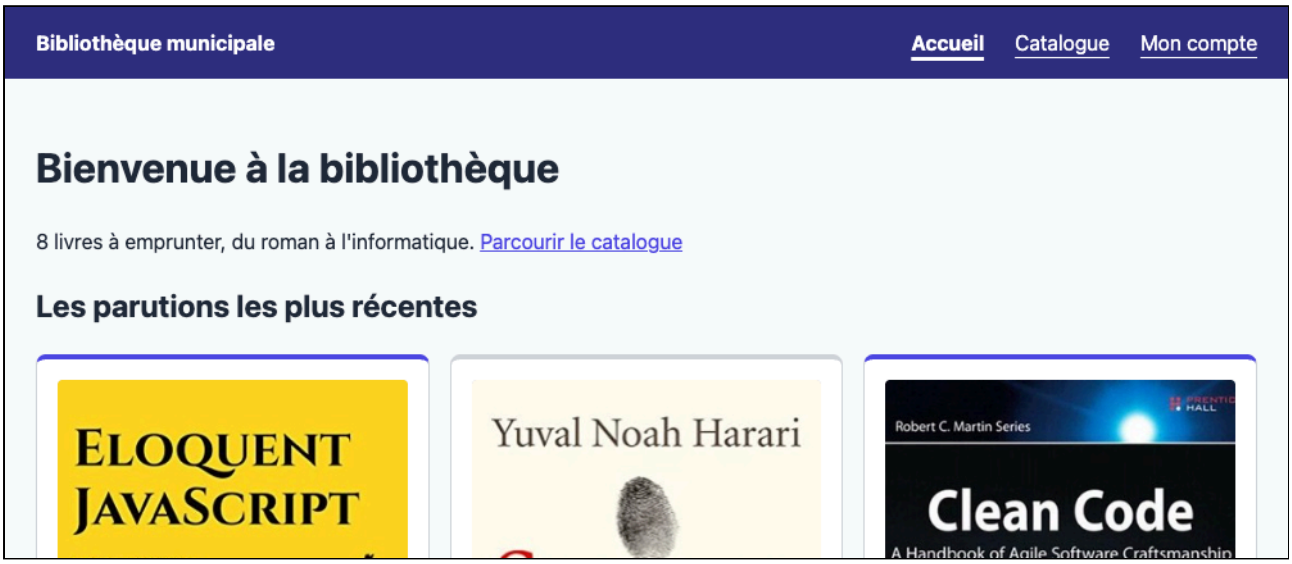
Dans `src/main.jsx`, remplacez :

```
      <App />
```

par :

```
<AuthProvider>
  <App />
</AuthProvider>
```

Vous devez voir : un `AuthProvider` au-dessus d'`App`, dans l'onglet *Components*. À l'écran, rien ne change, et c'est normal.

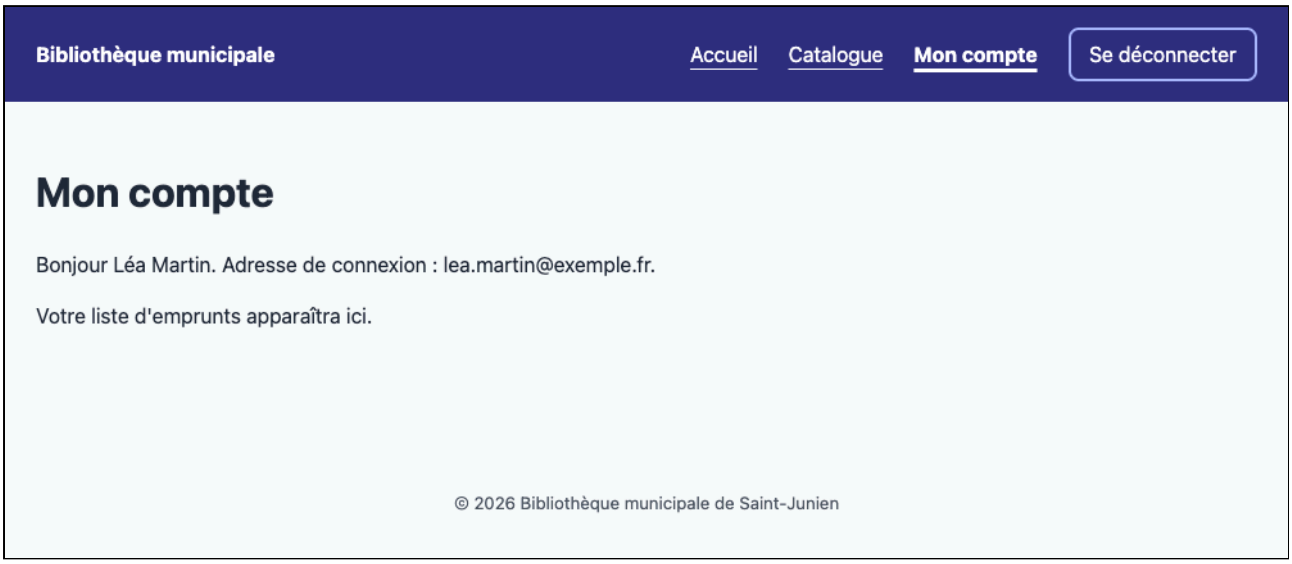


BIB-1 : l'application s'affiche comme avant

Livrez le ticket, puis relisez BIB-2.

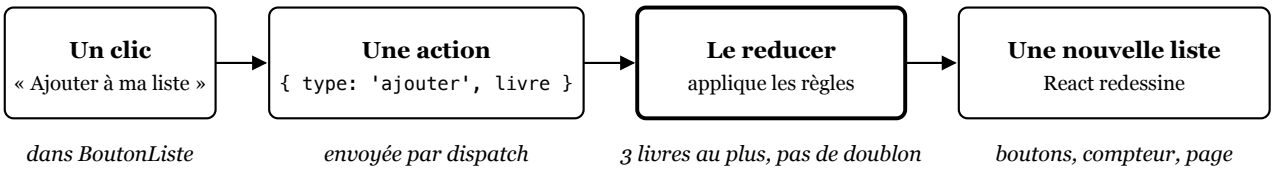
Ensemble, après les deux fusions

`git pull`, puis connectez-vous : `lea.martin@exemple.fr`, mot de passe `lecture`. La connexion marche, et « Mon compte » s'ouvre.



Fin du sprint 1 : Léa est connectée, le menu propose « Se déconnecter »

Sprint 2 · La liste



Ce que fait le reducer du sprint 2, à chaque clic

Le contrat : `useListe()` renvoie `{ liste, pleine, contient, ajouter, retirer, vider }`, et `MAXIMUM` vient de `listeReducer.js`.

BIB-4 · Le bouton, le compteur, la page

Branche : `bib-4-bouton-compteur-page`. Message : `BIB-4 : le bouton, le compteur et la page Ma liste`.

Les fichiers de `src/contexts/` sont des bouchons : la liste contient toujours *Clean Code*, et les clics ne changent rien.

Créez `src/components/BoutonListe.jsx` :

```
import { useListe } from '../contexts/ListeContext.jsx';

// Le même bouton sur les cartes et sur la fiche : il sait tout seul dans quel état
il est.
export default function BoutonListe({ livre }) {
  const { contient, pleine, ajouter } = useListe();

  if (!livre.disponible) {
    return <button type="button" disabled>Indisponible</button>;
  }
  if (contient(livre.id)) {
    return <button type="button" disabled>Dans ma liste</button>;
  }
  if (pleine) {
    return <button type="button" disabled>Liste pleine</button>;
  }
  return (
    <button type="button" onClick={() => ajouter(livre)}>
      Ajouter à ma liste
    </button>
  );
}
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `import { Link } from 'react-router';` :

```
import BoutonListe from './BoutonListe.jsx';
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `<p className="dispo">{livre.disponible ? 'Disponible' : 'Emprunté'}</p>` :

```
      <div className="actions">
        <BoutonListe livre={livre} />
      </div>
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { useAuth } from '../contexts/AuthContext.jsx';` :

```
import { useListe } from '../contexts/ListeContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `const { utilisateur, deconnecter } = useAuth();` :

```
  const { liste } = useListe();
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `<li><NavLink to="/catalogue">Catalogue</NavLink></li>` :

```
            <li>
              <NavLink to="/ma-liste">
                Ma liste <span className="pastille">{liste.length}</span>
              </NavLink>
            </li>
```

Créez `src/pages/MaListe.jsx` :

```
import { Link } from 'react-router';
import { useListe } from '../contexts/ListeContext.jsx';
import { MAXIMUM } from '../contexts/listeReducer.js';

export default function MaListe() {
  const { liste, retirer, vider } = useListe();
  const s = liste.length > 1 ? 's' : '';

  function confirmerVider() {
    if (window.confirm('Retirer tous les livres de votre liste ?')) {
      vider();
    }
  }

  return (
    <>
      <title>Ma liste — Bibliothèque municipale</title>
      <h1>Ma liste d'emprunts</h1>

      {liste.length === 0 ? (
        <p className="aucun">
          Votre liste est vide. <Link to="/catalogue">Choisir des livres dans le
catalogue</Link>
        </p>
      ) : (
        <>
          <p className="compteur">{liste.length} livre{s} sur {MAXIMUM} possibles.
</p>
          <ul className="selection">
            {liste.map((livre) => (
              <li key={livre.id}>
                <p><Link to={`/livres/${livre.id}`}>{livre.titre}</Link>,
{livre.auteur}</p>
                <button
                  type="button"
                  className="secondaire"
                  aria-label={`Retirer ${livre.titre}`}
                  onClick={() => retirer(livre.id)}
                >
```

```
                Retirer
              </button>
            </li>
          ))}
        </ul>
        <p className="barre">
          <button type="button" className="secondaire" onClick={confirmerVider}>
            Vider la liste
          </button>
        </p>
      </>
    )}
  </>
);
}
```

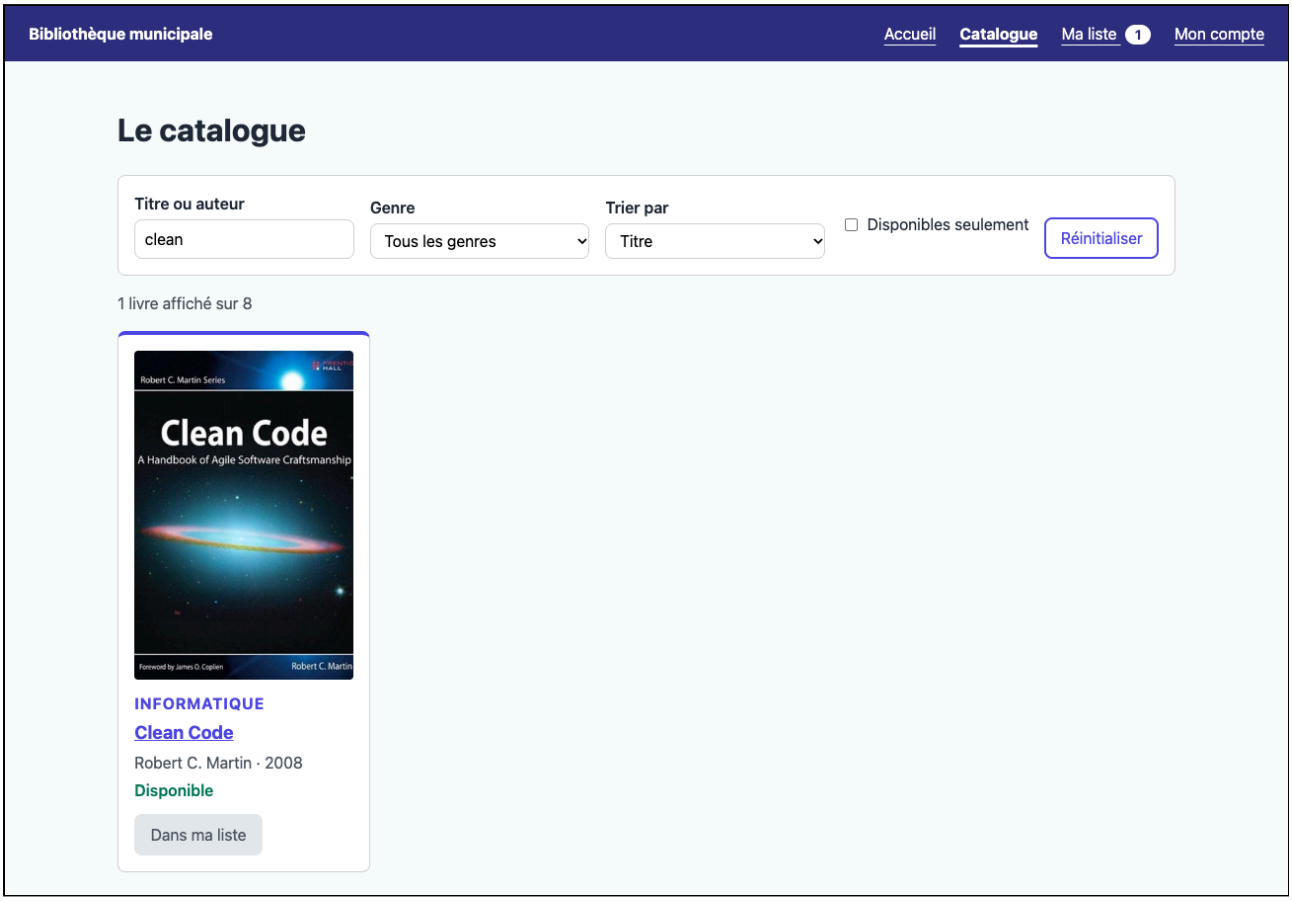
Dans `src/App.jsx`, juste après la ligne `import FicheLivre from './pages/FicheLivre.jsx';` :

```
import MaListe from './pages/MaListe.jsx';
```

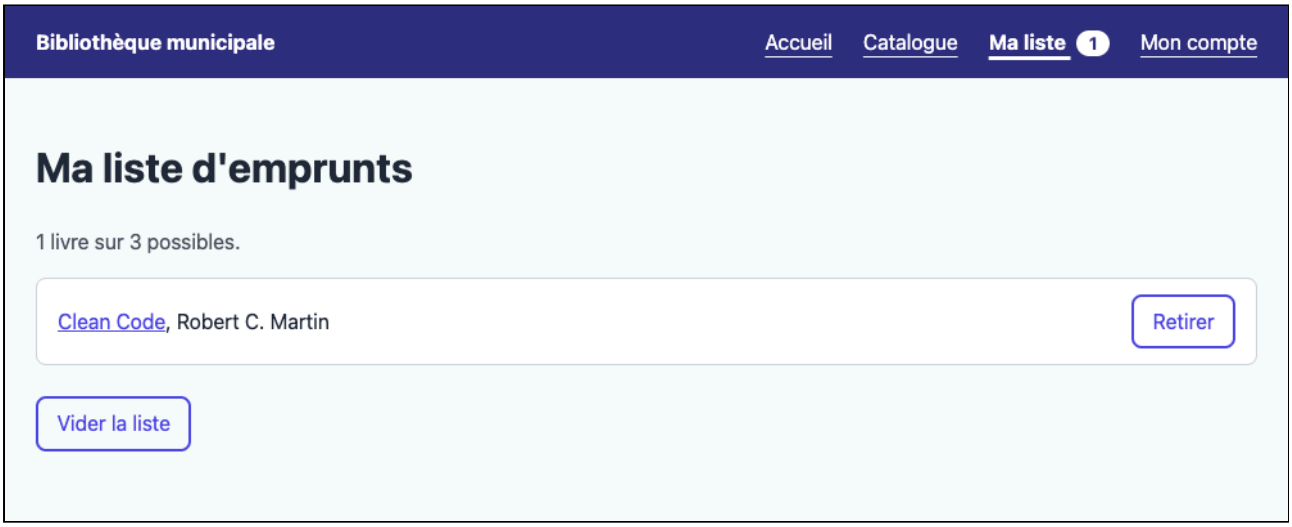
Dans `src/App.jsx`, juste après la ligne `<Route path="livres/:id" element={<FicheLivre />} />` :

```
      <Route path="ma-liste" element={<MaListe />} />
```

Vous devez voir : *Clean Code* « Dans ma liste », le compteur du menu à 1, et *Clean Code* dans la page « Ma liste ».



BIB-4, avec les bouchons : le bouton de Clean Code, et le compteur à 1

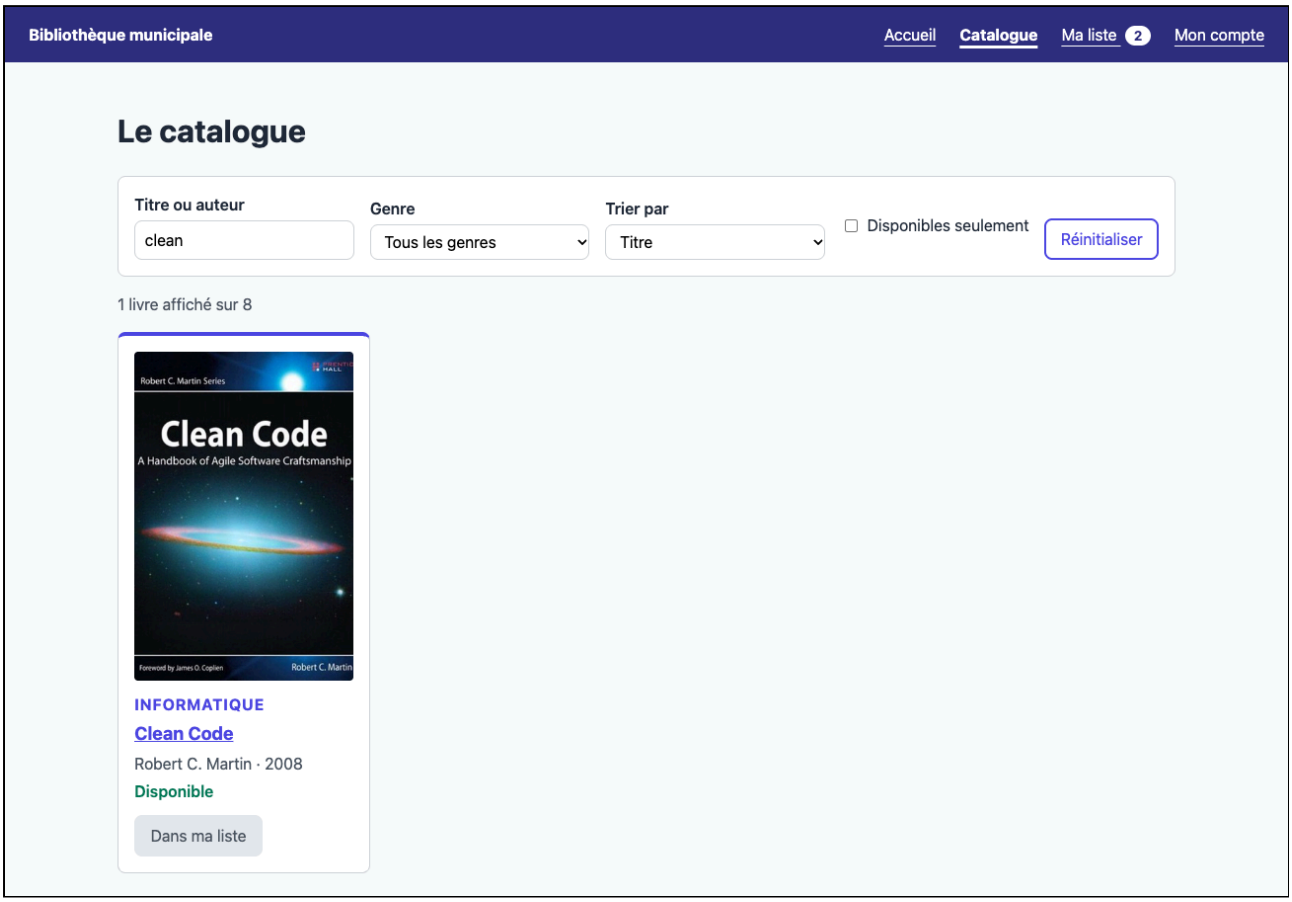


BIB-4, avec les bouchons : la page « Ma liste »

Livrez le ticket, puis relisez BIB-3.

Ensemble, après les deux fusions

`git pull`, puis ajoutez deux livres : le compteur du menu passe à 2. Dans « Ma liste », retirez-en un, puis videz la liste.



Fin du sprint 2 : deux livres ajoutés, le compteur dit 2

Pendant que B fait BIB-5

Lisez `src/contexts/ListeContext.jsx`, écrit par B : vous le modifiez au sprint 3. Puis relisez sa pull request.

Sprint 3 · La mémoire

BIB-6 · La liste survit au rechargement

Branche : `bib-6-memoire`. Message : `BIB-6 : la liste survit au rechargement`.

Dans `src/contexts/ListeContext.jsx`, remplacez :

```
import { createContext, useContext, useReducer } from 'react';
```

par :

```
import { createContext, useContext, useEffect, useReducer } from 'react';
```

Dans `src/contexts/ListeContext.jsx`, juste après la ligne `const ListeContext = createContext(null);` :

```
const CLE = 'liste-emprunts';

// Au premier affichage, on relit la liste enregistrée ; si elle est absente ou abîmée, liste vide.
function charger() {
  try {
    return JSON.parse(localStorage.getItem(CLE)) ?? [];
  } catch {
    return [];
  }
}
```

Dans `src/contexts/ListeContext.jsx`, remplacez :

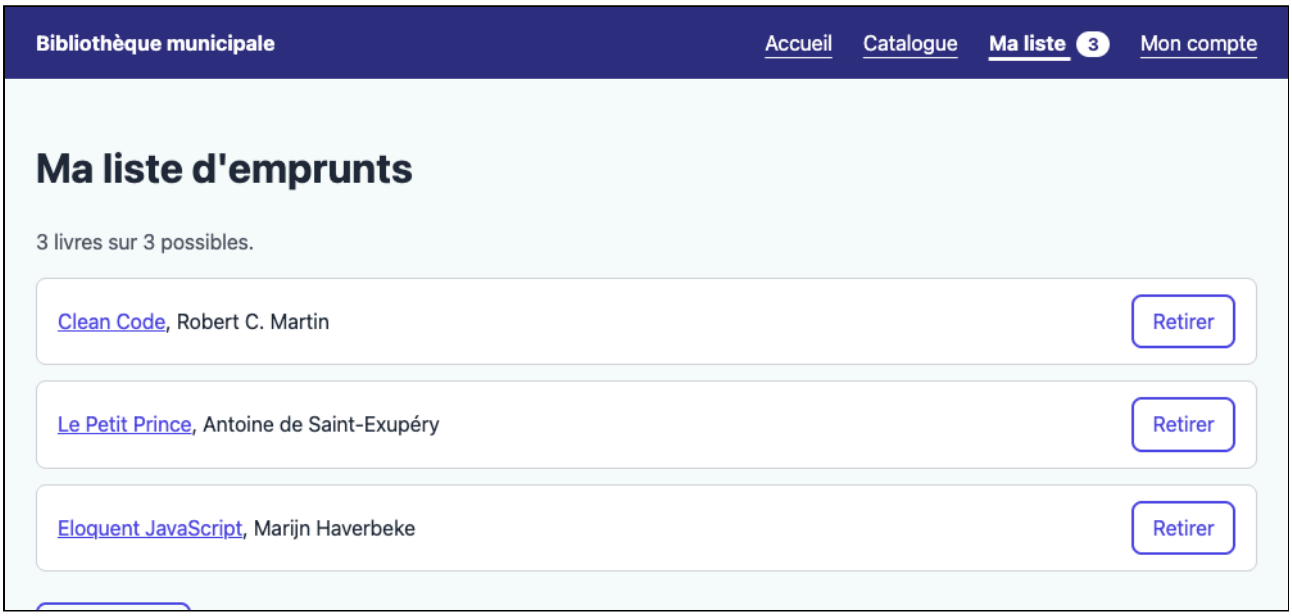
```
  const [liste, dispatch] = useReducer(listeReducer, []);
```

par :

```
const [liste, dispatch] = useReducer(listeReducer, [], charger);

// Après chaque changement de la liste, on l'enregistre.
useEffect(() => {
  localStorage.setItem(CLE, JSON.stringify(liste));
}, [liste]);
```

Vous devez voir : trois livres dans la liste, `F5`, ils sont toujours là.



BIB-6 : après un rechargement, la liste est intacte

Livrez le ticket, puis relisez BIB-7.

Ensemble, après les deux fusions

`git pull`. La liste survit à `F5`, et le compte l'affiche.

Rendre

Sur la page du dépôt GitHub : *Code*, puis *Download ZIP*. Renommez le fichier en `NOM1-NOM2-React-S3.zip`, et déposez-le une seule fois dans **Rendus — à déposer ici → Frameworks React → FM04 · Séance 3**.
